COMP1801

Executive Summary

In this report, a machine learning model is created to determine if a part has a sufficient lifespan for safe use using easily measurable factors. The dataset contains details about

Data Exploration

The dataset is loaded using the pandas library by importing it and it contains the details about different metal alloys and their properties. The dataset has 1000 rows and 16 columns, which means that it has 1000 values in each of the features. The column ‘Lifespan’ contains the hours of specific parts of metals, which ranges from 417.99 to 2134.53 and it is the target variable.

```
Summary of the data:
          Lifespan  coolingRate   quenchTime    forgeTime  HeatTr
count  1000.000000  1000.000000  1000.000000  1000.000000    1000
mean   1298.556320    17.639000     2.764230     5.464600      30
std     340.071434     7.491783     1.316979     2.604513      16
min     417.990000     5.000000     0.500000     1.030000       1
25%    1047.257500    11.000000     1.640000     3.170000      16
50%    1266.040000    18.000000     2.755000     5.475000      29
75%    1563.050000    24.000000     3.970000     7.740000      44
max    2134.530000    30.000000     4.990000    10.000000      59

           Nickel%        Iron%      Cobalt%    Chromium%  smallD
count  1000.000000  1000.000000  1000.000000  1000.000000   1000.
mean     60.243080    24.553580    12.434690     2.768650     17.
std       5.790475     7.371737     4.333197     1.326496     12.
min      50.020000     6.660000     5.020000     0.510000      0.
25%      55.287500    19.387500     8.597500     1.590000      7.
50%      60.615000    24.690000    12.585000     2.865000     18.
75%      65.220000    29.882500    16.080000     3.922500     26.
max      69.950000    43.650000    19.990000     4.990000     61.

       largeDefects  sliverDefects
count   1000.000000    1000.000000
mean       0.550000       0.292000
```

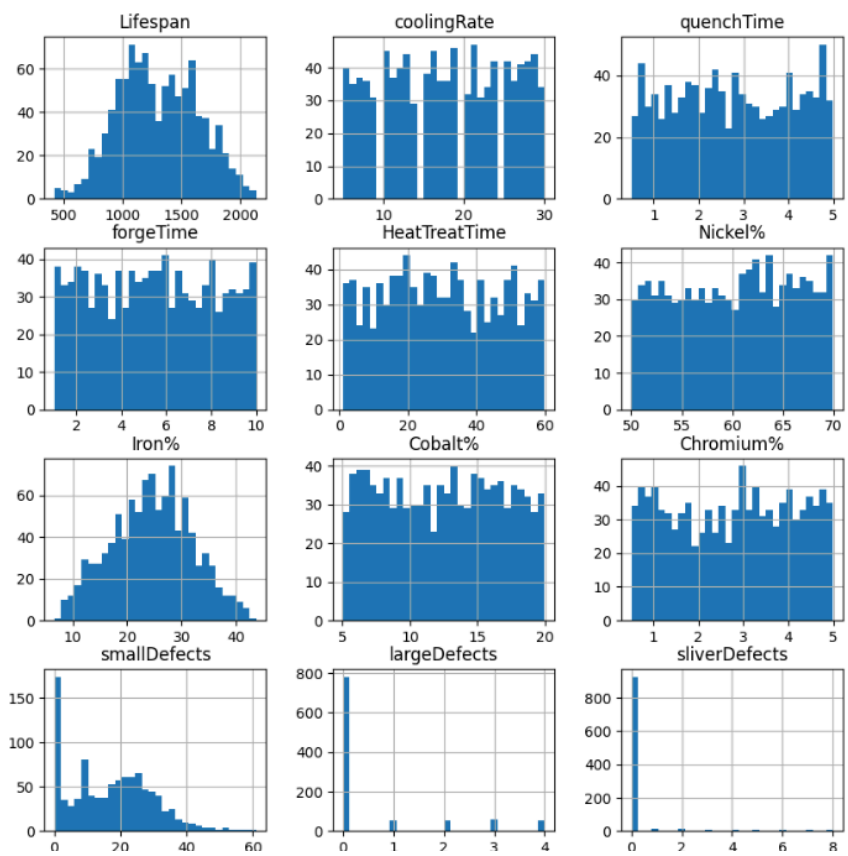
The other feature columns are namely, ‘partType’, ‘microstructure’, ‘coolingRate’, ‘quenchTime’, ‘forgeTime’, ‘HeatTreatTime’, ‘Nickel%’, ‘Iron%’, ‘Cobalt%’, ‘Chromium%’, ‘smallDefects’, ‘largeDefects’, ‘sliverDefects’, ‘seedLocation’ and ‘castType’. The ‘partType’, ‘microstructure’, ‘seedLocation’ and ‘castType’ are having object data type and all other columns are having numerical data type (float and integer). The dataset is already cleaned, so there is no need treat null values and missing values.

	Lifespan	partType	microstructure	coolingRate	quenchTime	forgeTime	HeatTreatTime	Nickel%	Iron%	Cobalt%	Chromium%	smallDefects	largeDefects	sliverDefects	seedLocation	castType
0	1469.17	Nozzle	equiGrain	13	3.84	6.47	46.87	65.73	16.52	16.82	0.93	10	0	0	Bottom	Die
1	1793.64	Block	singleGrain	19	2.62	3.48	44.70	54.22	35.38	6.14	4.26	19	0	0	Bottom	Investment
2	700.60	Blade	equiGrain	28	0.76	1.34	9.54	51.83	35.95	8.81	3.41	35	3	0	Bottom	Investment
3	1082.10	Nozzle	colGrain	9	2.01	2.19	20.29	57.03	23.33	16.86	2.78	0	1	0	Top	Continuous
4	1838.83	Blade	colGrain	16	4.13	3.87	16.13	59.62	27.37	11.45	1.56	10	0	0	Top	Die
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	1217.50	Block	singleGrain	5	1.91	8.55	34.76	68.11	20.73	7.15	4.01	0	0	0	Bottom	Investment
996	742.05	Block	singleGrain	10	0.67	7.99	46.21	51.25	32.12	12.23	4.40	1	0	0	Bottom	Investment
997	1448.26	Nozzle	equiGrain	17	3.12	7.61	59.87	54.09	33.20	9.31	3.40	16	1	0	Bottom	Investment
998	1099.10	Valve	colGrain	23	4.48	2.39	17.72	50.32	31.92	13.01	4.75	11	0	0	Top	Continuous
999	1086.82	Valve	colGrain	10	3.52	2.29	40.10	66.75	12.08	17.89	3.28	9	3	0	Top	Continuous

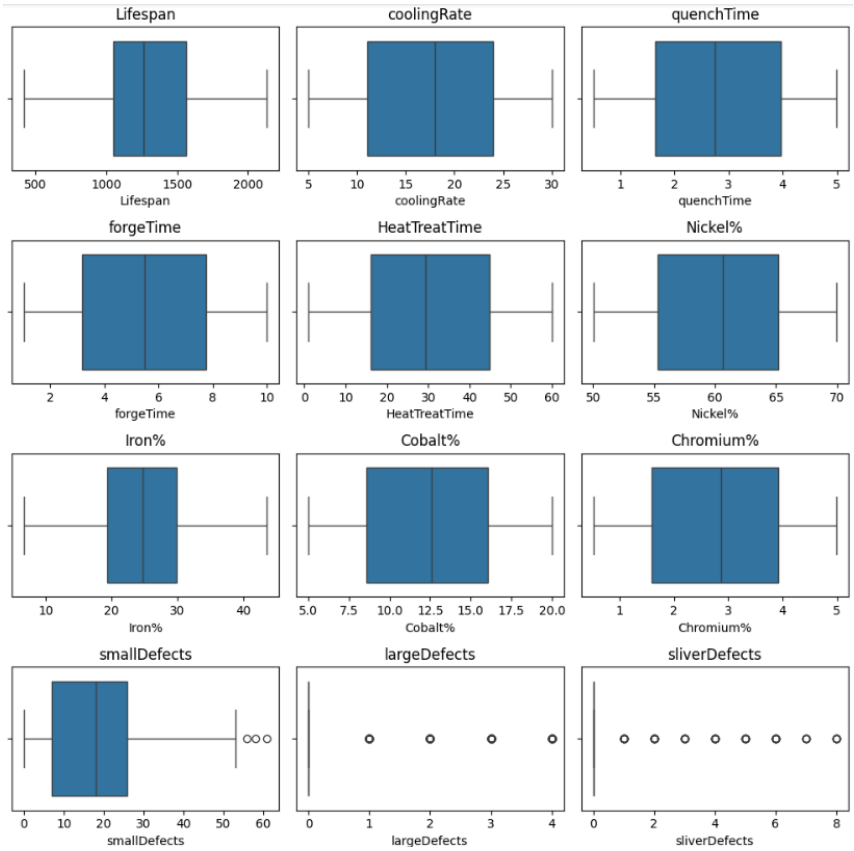
1000 rows × 16 columns

Now, since there are some categorical columns in the dataset, the numerical columns are separated by putting them into a list to use them for visualizing some graphs, which helps to find out some meaningful insights

When a histogram is plotted, it is observed that ‘Lifespan’ and ‘Iron%’ is having a bell shaped distribution. Features like ‘CoolingRate’, ‘ForgeTime’, ‘HeatTreatTime’, and ‘Nickel%’ are having a uniform distribution meaning that their values are evenly distributed across the range. Features such as ‘smallDefects’, ‘largeDefects’ and ‘sliverDefects’ are highly right skewed which indicates that there are near to no values of these defects to the rear end of the range.



The main objective for plotting a box plot is to check if there are any outliers in the dataset. When it is plotted, some outliers in ‘smallDefects’, ‘largeDefects’ and ‘sliverDefects’ were found.



Regression Implementation

Methodology

Encoding is the process of converting categorical columns (object data type) into numerical columns because the machine only understands numerical values. Since the histogram and box plot does not give any meaningful information for the categorical features even if encoded, it was not encoded for those plots. But encoding is required to plot a correlation matrix since relation between categorical features is also important.

There are mainly two encoding methods, namely label encoding and one hot encoding. Label encoding will be used when a clear order or ranking is required to be preserved whereas one hot encoding will be used when the order of the categories does not have any importance. One hot encoding method was chosen over label encoding to encode the categorical columns, because these column does not require a clear order or ranking for the purpose of encoding.

Firstly, ‘partType’ column was encoded into 4 differents columns, namely ‘partTypeNozzle’, ‘partTypeBlock’, ‘partTypeBlade’, ‘partTypeValve’. Then, ‘microstructure’ column was encoded

into 3 different columns, namely ‘microstructureEquiGrain’, ‘microstructureSingleGrain’ and ‘microstructureColGrain’. The ‘seedLocation’ column was encoded into 2 different columns, namely ‘seedLocationTop’ and ‘seedLocationBottom’. Finally, ‘castType’ column was encoded into 3 different columns, namely ‘castTypeDie’, ‘castTypeInvestment’ and ‘castTypeContinuous’. Each column will be having the values 0’s and 1’s, 1 when the value is true or 0 if not.

A correlation matrix is now plotted and observed that ‘coolingRate’ and ‘smallDefects’ are having positive high co-relation, ‘Iron%’ and ‘Nickel%’ was having high negative correlation, ‘Iron%’ and ‘Cobalt%’ was having a moderate negative correlation and all the encoded features are having moderate to high correlation to each of their own features. So, there exists a multicollinearity between these features.

Scalar standardization was done on the encoded data to convert the numerical values into a standard range for between the features.

To eliminate the multicollinearity, Principal Component Analysis (PCA) was introduced. PCA was used to reduce the dimensionality of data sets, resulting in a smaller collection of variables while maintaining important set information. Here, the PCA was implemented on each of the above-mentioned correlations and gave the new column names as ‘PCA_1’, ‘PCA_2’, ‘PCA_3’, ‘PCA_4’, ‘PCA_5’ and ‘PCA_6’ respectively.

The resulting correlation matrix plotted had a negative moderate correlation between ‘Cobalt% and ‘PCA_2’, and the ‘sliverDefects’ and ‘PCA_4’. So the PCA was implemented again a couple of times and gave the new column names as ‘PCA_C_2’ and ‘PCA_s_4’ respectively, which resulted in a balanced correlation matrix plot.

The predictor variable is finalised from the cleaned correlation matrix. The ‘quenchTime’, ‘HeatTreatTime’, ‘PCA_1’, ‘PCA_3’ and ‘PCA_C_2’ was chosen as the predictor variables since they have relatively higher correlation with ‘Lifespan’ than the other columns (almost more than 0.1 values). The target variable was set as ‘Lifespan’ since their values are predicted by other values.

Now the data is split into 2 sets, which are the training set and the testing set. 80 percent of the data is loaded into the training set and the remaining 20 percent of the data was loaded into the testing set. The data in the training set will be used to train the model and the data in the testing set will be used to predict the Lifespan values using the trained model.

Selected Models

Polynomial Regression and Random Forest Regression was modelled for predicting the target variable ‘Lifespan’.

Polynomial Regression

In the polynomial regression, polynomial features and linear regression models was combined together in a pipeline. Hyperparameter tuning was also performed on the model using GridSearchCV. Grid Search parameters were set, which were poly__degree that specifies the degree of the polynomial features to be tested, linear__fit_intercept which determines whether the regression model should include an intercept term and linear__n_jobs which controls the number of parallel jobs for fitting the model.

This model was chosen because it was having the highest r2 score among other which were tested. Beside that, it is the best model for capturing non-linear relationships between the predictor and the target variable.

Random Forest Regression

In the Random Forest Regression, the model fitting was also performed with GridSearchCV hyperparameter tuning. The Grid Search parameters were set, which were n_estimators which depicts the number of decision trees in the forest, max_depth which depicts the maximum depth of each tree,min_samples_split which depicts the minimum number of samples required to split an internal node, min_samples_leaf which depicts the minimum number of samples required to be at a leaf node, n_jobs which depicts the number of parallel threads to use for training the forest and random_state which sets the seed for reproducibility.

This model was chosen because it was having the second highest r2 score among others that min_samples_split were tested. Besides that, Random Forest regression is a robust and versatile model that uses an ensemble of decision trees to make predictions.

Evaluation

When comparing both the models which were fitted, the r2 score and RMSE of the Polynomial Regression outperformed Random Forest Regression.

Regression Models	**R2 Score**	**RMSE**
Polynomial Regression	0.66	0.35
Random Forest Regression	0.64	0.37

The best hyperparameter using GridSearchCV was also found out for each of the models.

Hyper parameters	**Value**
poly__degree	4
linear__fit_intercept	False

linear__n_jobs	1

A poly__degree of 4 suggests that the relationship between the features and the target variable is complex and nonlinear. A linear__fit_intercept of False means the intercept term is not included. A lineat__n_jobs of 1 means that the computation was performed sequentially.

Hyper Parameter	**Value**
n_estimators	500
max_depth	10
min_samples_split	10
min_samples_leaf	4
n_jobs	1
random_state	42

A n_estimator of 500 was giving improved model performance by providing robustness and better dataset representation. A max_depth of 10 was most likely complex enough to require deeper trees instead of shallow trees, which helps to prevent overfitting. A min_samples_split of 10 suggests that the model avoids branching for very small sections of data. A min_samples_leaf of 4 denotes that it generalized leaf nodes rather than overly fine partitions. A n_jobs of 1 suggests the computation was performed sequentially. A random_state of 42 is a standard practice for ensuring consistent model behavior.

Critical Review

Classification Implementation

Feature Crafting

The original dataset is again loaded. Now, an additional binary feature for determining whether a part is defective or not was created as per the requirement of the company, namely ‘1500_labels’. Value 1 will denote that the corresponding metal part is defective and the value 0 will denote otherwise. Now this new feature is the target variable for implementing the classification models. The ‘Lifespan’ column is dropped from the dataset since it is not required anymore.

Methodology

All the preprocessing steps that was implemented for the regression model was again done on this new dataset, which were categorical feature encoding, feature scaling, Principal Component Analysis (PCA) and splitting the data into training and test sets.

The predictor variable was finalised from the cleaned correlation matrix. The 'quenchTime', 'PCA_3' and 'PCA_C_2' was chosen as the predictor variables since they have relatively higher correlation with '1500_labels' than the other columns (almost more than 0.1 values). The target variable was set as '1500_labels' since their values are predicted by other values.

Selected Models

Logistic Regression Classifier and XGBoost Classifiers was modelled for predicting the target variable '1500_labels'.

Logistic Regressor Classifier

In the Logistic Regressor Classifier, the model fitting was performed with GridSearchCV hyperparameter tuning. The Grid Search parameters were set, which were 'C' which is the inverse of regularization strength, 'penalty' which specifies the type of regularization applied to the model and 'solver' which determines the optimization algorithm used to fit the Logistic Regression model.

The reason for choosing Logistic regressor classifier is that it is computationally efficient and can scale to handle big datasets. It works especially well when the connection between the features and the target variable is roughly linear.

XGBoost Classifier

In the XGBoost Classifier, the model fitting was also performed with GridSearchCV hyperparameter tuning. The Grid Search parameters were set, which were 'n_estimators' which represents the number of trees in the ensemble, 'max_depth' which specifies the maximum depth of each decision tree in the ensemble, 'learning_rate' which determines the step size at each boosting iteration, 'subsample' which specifies the fraction of the training data to sample for each boosting round and 'colsample_bytree' which specifies the fraction of features (columns) to sample for each tree.

XGBoost gives a wide range of hyperparameters that can be fine-tuned to improve model performance.

Evaluation

When comparing both the models which were fitted, the accuracy score and classification report of the Logistic Regressor Classifier and XGBoost Classifier were evaluated.

Both the models was having the same accuracy, which was 0.72.

The best hyperparameter using GridSearchCV was also found out for each of the models.

The best hyperparameters for Logistic Regressor Classifier is given below.

Hyper parameters	Best Value
C	0.1
penalty	l2
solver	saga

C=0.1 indicates that the model performs best with moderate regularization, penalty=l2 refers to Ridge Regularization and solver=saga was used as it works well with l2 penalty and big data can be handled.

The best hyperparameters for XGBoost Classifier is given below.

Hyper parameter	Value
colsample_bytree	0.8
learning_rate	0.01
max_depth	3
n_estimators	150
subsample	1.0

colsample_bytree=0.8 means that the model introduce randomness using 80 percent of the features, learning_rate=0.01 means slower learning of the estimators, , max_depth=3 means the tree will be staying shallow, n_estimators=150 means that it have a balance between performance and efficiency and subsample=1.0 means that all samples are used for all tree.

Conclusion

- In the data exploration of the dataset, some features were highly right skewed and there existed outliers in those features.
- The dataset provided had both categorical and numerical features, which required one hot encoding to convert the categorical values into numerical.
- Multicollinearity was eliminated using Principal Component Analysis (PCA).
- Two regression models were modelled to predict ‘Lifespan’ feature, namely Polynomial Regressor and Random Forest Regressor.
- Polynomial Regressor had a r2 score of 0.66 while Random Forest Regressor had a r2 score of 0.64.
- Two classifier models were modelled to predict the newly created ‘1500_label’ feature, namely Logistic Regressor Classifier and XGBoost Classifier.
- Both Logistic Regressor Classifier and XGBoost Classifier had an accuracy of 0.72.